

AccUDNN: A GPU Memory Efficient Accelerator for Training Ultra-deep Neural Networks

Jinrong Guo^{1,2}, Wantao Liu^{1*}, Wang Wang^{1,2}, Chunrong Yao³, Jizhong Han¹,
Ruixuan Li¹, Yijun Lu⁴, Songlin Hu^{1*}

¹*Institute of Information Engineering, Chinese Academy of Sciences*

²*School of Cyber Security, University of Chinese Academy of Sciences*

³*School of Computer Science and Engineering, Nanjing University of Science and Technology*

⁴*Alibaba Cloud Computing Co. Ltd.*

Beijing, China

{guojinrong, liuwantao, wangwang, hanjizhong, liruixuan, husonglin}@iie.ac.cn,

yaochunrong@njnu.edu.cn, yijun.lyj@alibaba-inc.com

Abstract—With the implementation of mainstream DL frameworks, scarce GPU memory resource is the primary bottleneck that hinders the trainability and training efficiency of ultra-deep neural networks (UDNN). Prior memory optimization works focus on removing the trainability restriction but leave the training efficiency out of consideration. To fill the gap, we present "AccUDNN", an accelerator that aims to make full use of finite GPU memory resource to speed up the training process of UDNN in this paper. AccUDNN mainly includes two modules: memory optimizer and hyperparameter tuner. Memory optimizer develops a novel performance-model guided dynamic swap out/in strategy to meet trainability first and further remedy the efficiency degradation in other swapping strategies. Then, a hyperparameter tuner is designed to explore the efficiency-optimal minibatch size and the matched learning rate after applying the dynamic swapping strategy. Evaluations demonstrate that AccUDNN cuts down the GPU memory requirement of ResNet-152 from more than 24GB to 8GB. In turn, given 12GB GPU memory budget, the efficiency-optimal minibatch size can reach 4.2x larger than Caffe and finally improve the scaling efficiency (speedup) of 8 GPUs' cluster by 1.9x.

Index Terms—ultra-deep neural network, GPU, memory optimization, speed up

I. INTRODUCTION

Scale of data volume and computational infrastructure together make current deep learning flourish, especially in computer vision [1], [2] and NLP [3], [4] fields. From 8-layer AlexNet [1] to 152-layer ResNet [2], the neural network architectures get deeper and the corresponding model quality gets better. However, training a hundreds-of-layers UDNN, as ResNet or BERT [3], is a resource-intensive task, involving not only a significant number of computing resources but also huge memory space. Nowadays, GPU's multi-core parallel architecture along with high memory bandwidth makes it a popular choice for training DNN model. But regrettably, the limited size of GPU memory, generally at most 24GB, is far from sufficient to accommodate UDNN, leading to either out-of-memory or a very small minibatch size.

The data occupying memory of UDNN include model parameter, feature map, their gradients, pre-cached input sam-

ples, and necessary workspace, etc. Feature map is the intermediate state that is generated in the forward propagation and reused for gradients calculation in the backward propagation. The memory footprint of feature map is not only related to the depth of UDNN but also increases proportionally with the minibatch size. Together with its gradient, feature map takes up a large percentage of total memory requirement, even up to 95%. Thus, we mainly target to the feature map in this paper.

In the design of current DL frameworks, they put all the involved data in the training process in GPU memory. To handle UDNN, we must decrease the minibatch size or adopt model parallelism mode. Indeed, these two approaches are able to break the trainability restriction, but the negative effects of underutilization of GPU computing resources and heavy distributed communication overhead damage the training efficiency. However, in practical use, in addition to the trainability, we should further consider the training efficiency, which is crucial to make UDNN truly applicable. For example, Y. You points out that finishing 100-epoch ImageNet-1k training with ResNet-50 on a single NVIDIA M40 GPU takes 14 days [5]. Such a long training time destroys the interactivity and limits productivity severely.

At present, distributed training has become a popular scheme to accelerate the training process and data parallelism mode is widely adopted [6]. When employing distributed data parallelism mode to train UDNN, if the memory-restricted minibatch size of UDNN on single GPU gets very small, more frequent parameter synchronizations among cluster would be required to finish a fixed number of epochs. As a consequence, the computation-to-communication (comp-to-comm) ratio decreases, so does the scaling efficiency. In extreme case, no speed-up achieved by distributed cluster is not impossible.

To address both the challenges in trainability and training efficiency of UDNN, this paper presents AccUDNN accelerator. AccUDNN incorporates two core modules: memory optimizer and hyperparameter tuner. Firstly, by identifying the trainability limitation of current DL frameworks and the performance degradation of the prior swap out/in optimization approaches [7], [8], our memory optimizer develops a dynamic

* Corresponding author

swap out/in strategy between GPU and host memory to overcome the trainability restriction in a way without damaging the training efficiency. The memory optimizer achieves this by dynamically orchestrating which feature map should be swapped to host side and when to swap according to the attributes of specific UDNN architecture and hardware resources, hiding the communication of swapping operation behind GPU computation. Then, after applying the swapping strategy, the runtime environment gets complicated with host memory and PCIe communication involved, it gets difficult to determine the efficiency-optimal hyperparameter. Thus, the hyperparameter tuner is further designed to fix this dilemma.

In brief, AccUDNN first makes the UDNN trainable and maintains the baseline efficiency by smartly overlapping the communication of swapping operation with GPU computation, then, AccUDNN further explores the efficiency-optimal hyperparameter to unleash the better performance of single GPU and distributed cluster to finally accelerate the training process.

Our contribution. Compared with prior works, AccUDNN highlights in the following three aspects.

1) The memory optimizer is the first one that can smartly pick out those suitable feature maps to swap by the aid of performance model. It eliminates GPU computation stall (i.e. performance degradation) in prior heuristic-based works at the same time of meeting trainability.

2) The hyperparameter tuner is first developed from a system perspective to explore the optimal training efficiency after applying the memory optimization technique. Compared with prior works that end up with the elementary conclusion of saving GPU memory or increasing minibatch size, we step further and take the training efficiency into account.

3) AccUDNN takes both system optimization and algorithmic improvement into consideration. It provides a complete and transparent solution to train UDNN, more than relieving the pressure on GPU memory.

II. RELATED WORK

Existing memory optimization techniques for single GPU include recomputation [9], data encode compression [10], swap out/in strategy between GPU and host memory [7], [11], [12]. Chen [9] proposes the idea of trading computation for memory, dropping partial feature maps in forward propagation and recomputes them when needed in backward propagation. Gist [10] takes a slightly different approach, they store those feature maps in an encoded representation and decode them when used again. vDNN [7] utilizes the relatively abundant host memory as auxiliary storage for GPU and swaps feature maps out to host side. In addition, Wang makes a combination of recomputation and swap operation, achieving a greater degree of memory reduction [8]. These techniques do reduce GPU memory consumption, but they all give rise to performance degradation from 4-30%. For example, in the swapping technique, this is usually caused by the insufficient overlap of PCIe data transfer and GPU computation when the minibatch size is set as the maximal one in GPU memory-optimal state.

For hyperparameter minibatch size selection, increasing interest has turned to large minibatch size to speed up the training process [13], [14]. For a single GPU, large minibatch size enables higher parallelism of GPU computing units [15]. In the distributed cluster, large minibatch size is conducive to scale data parallelism mode across multiple GPUs. Two approaches to increase minibatch size are: use more GPUs and increase minibatch size on per GPU.

The former approach has already explored by IT companies with abundant hardware resources. Facebook's experiment [16] employs 256 GPUs to train ResNet-50 with a minibatch size of 8192 and is completed in 1 hour. Akiba further extends Facebook's experiment to 1024 GPUs and finishes it in 15 minutes [13]. However, the later approach hasn't been fully studied due to the limitation of GPU memory capacity, in the above-mentioned two experiments, minibatch size on per GPU is set as 32 only. Given a fixed number of servers, the smaller minibatch size processed by each server is, the more frequent global parameter synchronizations among cluster are. With common commodity Ethernet, the time cost of inter-machine communication among dozens or hundreds of servers usually leads to diminishing returns.

In a nutshell, how to fully exploit the available hardware resource with co-design of optimization technique to train UDNN within a reasonable time range is still pending to be solved.

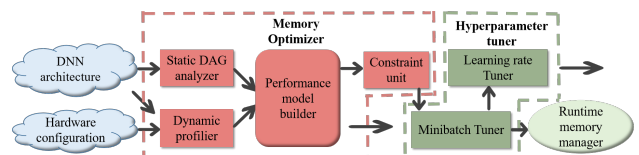


Fig. 1: AccUDNN architecture

III. OVERALL ARCHITECTURE

This section gives an overview of AccUDNN accelerator by walking through the core components, as shown in Figure 1. Given a certain UDNN and hardware configuration, the information collector first gathers both the static and dynamic attributes that are required to establish the performance model. Through data fitting, performance model builder traces the basic runtime behavior of the swap out/in strategy, including GPU computational performance, GPU memory usage, and PCIe communication performance. Then, the constraint unit sketches the restrictive relation among these three submodels and extracts the constraint condition that does not incur performance degradation in our dynamic swap out/in strategy. By integrating the constraint with performance model, hyperparameter tuner converts the training process after applying the dynamic swapping strategy into an optimization problem and finally figures out the efficiency-optimal hyperparameter minibatch size, then, learning rate changes adaptively. Once the optimal minibatch size is determined, the instantiated swap out/in strategy (what to be swapped and when to swap) that meets the performance constraint gets clear as well. Finally,

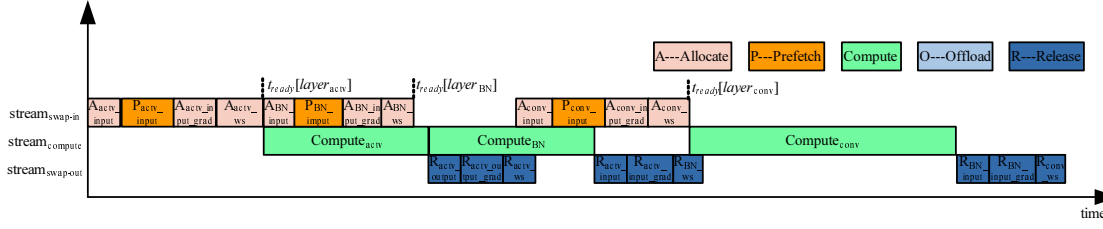


Fig. 2: Backpropagation with naive swap out/in operation

the concrete strategy is submitted to the runtime memory manager to deploy and execute.

IV. MEMORY OPTIMIZER

Given a certain UDNN and available hardware configuration, the primary thing to consider is the trainability, once it is trainable, the optimal training efficiency is expected. The memory optimizer develops a dynamic technique to seek the optimal training scheme. In detail, when the GPU memory is very scarce for an UDNN, it would be mandatory to swap all the involved data out to host memory to attain trainability, despite the training efficiency in this case might be unsatisfying. But usually, the GPU memory is somewhat limited (but not very scarce), it's no longer necessary to take the extreme approach. Once the trainability is ensured, we can shift the focus to the training efficiency. For training efficiency, there is a delicate balance. Swapping out as much data as possible enables larger minibatch size to be processed, which is beneficial to improve efficiency by fully utilizing GPU computing resources and reducing parameter synchronization among cluster, however, if the total volume to be transferred exceeds the PCIe bandwidth capacity, GPU computation would be stalled. Thus, memory optimizer is dedicated to figure out the optimal training mode between trainability and training efficiency by dynamically adjusting the swapping strategy.

A. Design principle

For convenience, some prerequisites are given first:

- 1) a N -layer deep neural network is unfolded as $2N$ forward propagation processes, j denotes layer index and $j \in [1, 2N]$.
- 2) due to the properties of high-ratio memory occupancy and the long time interval between two uses, the memory optimizer mainly targets feature maps.
- 3) four basic operations involved in swap out/in strategy: allocate, release, offload, prefetch. Offload refers to the swap-out operation from GPU to host memory and prefetch runs in the reversed direction.

In the beginning, we take the backpropagation process of typical convolutional unit "conv-BN-actv" as example to give a direct view of the execution mode after incorporating the naive swap out/in strategy (swap out all the feature maps to host side), as vDNN_{all} mode [7]. Figure 2 demonstrates the most common scenario. Because the model parameters and their gradients reside in GPU memory, the access overhead

is negligible, we don't display them explicitly in Figure 2. $t_{ready}[j]$ is the timestamp that marks all the data required by the computation of layer j have been swapped in GPU memory and the necessary workspace has been allocated, before this point, GPU cannot execute layer j 's computation and enters into the stall state. It is observed that the computation of BN layer follows actv layer compactly while an undesirable stall lies between BN and conv layer, caused by the inefficient overlapping of prefetch operation P_{conv_input} and BN's fast computation. In this case, the input feature map of conv layer is not suitable to swap out in terms of efficiency.

The flow chart of our dynamic swap out/in strategy is shown in Figure 3. Two key steps are highlighted here.

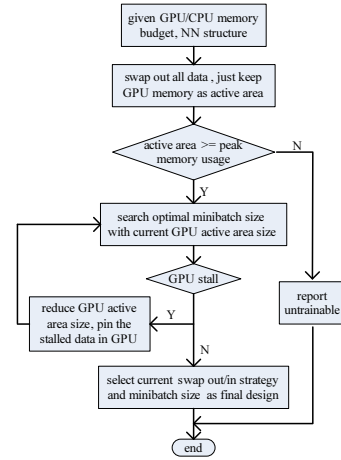


Fig. 3: Flow chart of dynamic swap out/in strategy

First, to maximize the trainability, the extreme case is to swap all the feature maps out to host side, just leaving GPU memory as layer-wise active area for necessary data and workspace accessed by current layer's computation. A peak layer-wise memory usage can be obtained by traversing the allocate/release operations in the global memory-object access pattern (GMAP) and it is proportional to the minibatch size. As long as the GPU memory budget is larger than the peak layer-wise memory usage, the UDNN is trainable and the processable minibatch size reaches its maximum at this point. The reduction of GPU memory usage from network-wide to layer-wise removes original depth limitation and enables much larger minibatch size.

Then, if the trainable condition is satisfied, the memory optimizer moves forward to explore the efficiency-optimal swap out/in strategy. We employ a performance model to exam whether the data volume to be transferred with current minibatch size will stall GPU computation or not, if it does, we shrink the minibatch size gradually, freeing up some GPU memory from the active area to pin those stalled feature maps and won't swap them out anymore. By adjusting gradually, the expected swap out/in strategy with no efficiency degradation can be found finally.

B. Information gathering

To build the performance model, information collector needs to capture static and dynamic information respectively, to profile the specific characteristics of the given UDNN and hardware configuration.

Static information. For a given UDNN with user-specified minibatch size k_{base} , by a static traverse of the dataflow graph, we can collect:

1) global memory-object access pattern (GMAP), stacked by four basic swap out/in operations, denotes the data access and release pattern within one iteration. The GMAP is invariable across iterations.

2) data size of every operation in GMAP, equals to memory consumption and PCIe transfer volume. The size of feature maps related operation is proportional to minibatch size.

3) single precision floating-point operations (FLOPs) of per layer, similar to 2), it grows linearly with the minibatch size. Both 2) and 3) can be calculated by simple analysis of tensor dimension, besides, the results under arbitrary minibatch sizes can be figured out easily with record of base minibatch size and the proportional relation instead of starting from scratch.

Dynamic information. We use a costless profiling stage to gather dynamic information, in detail, we set minibatch size as 1/8, 1/4, 1/2, 2/3, and 1 of the maximal trainable minibatch size respectively in the first five epochs and employ the naive swap out/in strategy without considering the stall problem. In the runtime of these five epochs, we collect:

1) per layer's computation time in one iteration of different minibatch sizes.

2) per swap operation's PCIe transfer time in one iteration of different minibatch sizes.

The dynamic information collection is conducted by *nvprof* profiling tool. The profiling approach benefits in three aspects. First, it can model various hardware configurations (GPU architecture, PCIe bandwidth) automatically without any user intervention. Furthermore, the results are much closer to practical condition compared with the nominal values, which profits the effective design of our swapping strategy. Lastly, integrated as early epochs of the whole training process, the profiling stage is still contributory.

C. Performance model

The first key point in the memory optimizer is to identify whether and when GPU computation stalls. We develop a performance model to handle this in a quantitative manner.

The performance model includes three parts: GPU computation model, GPU memory usage model, PCIe communication model.

GPU computational model. GPU computational performance for given UDNN is first modeled. All the collected information tuples, per layer's FLOPs and computation time, among five iterations with different minibatch sizes are categorized to multiple groups by layer type (conv, BN, pooling, FC, etc.). Then, the computing performance for each layer type is modeled respectively by data fitting. But the general performance trend of all the layer types is consistent, the throughput FLOPS grows gradually within a certain range and eventually saturates. The saturation point identifies the maximum available degree of parallelism, exceed which the execution of a kernel with extreme large FLOPs will be converted to serial steps and each step will use up all the computing resources. The FLOPS at saturation point of each layer type may differ from the others', which depends on their kernel implementation.

By querying the corresponding FLOPS from the performance curve, it is handy to calculate a layer's computation time under arbitrary minibatch size k . Then, the execution time of one iteration and the whole training process can be modeled successively.

Formally, given a N -layer UDNN, layer index j , minibatch size k , a constant number of training epochs E , dataset size m . Per layer's floating-point operation is defined as $FLOPs_j(k)$. Based on static information 3), we have

$$FLOPs_j(k) = \frac{k}{k_{base}} FLOPs_j(k_{base}), j \in [1, 2N] \quad (1)$$

With per layer's computation time $t_j(k)$, execution time for one iteration $t_{iter}(k)$ approximately equals to the sum of $t_j(k)$ due to the elimination of GPU computation stall in our swapping strategy, thus

$$\begin{aligned} t_{iter}(k) &= \sum_{j=1}^{2N} t_j(k) = \sum_{j=1}^{2N} \frac{FLOPs_j(k)}{FLOPS(FLOPs_j(k))} \\ &= \sum_{j=1}^{2N} \frac{\frac{k}{k_{base}} FLOPs_j(k_{base})}{FLOPS(\frac{k}{k_{base}} FLOPs_j(k_{base}))} \end{aligned} \quad (2)$$

and execution time for the whole training process $t_{whole}(k)$ is

$$\begin{aligned} t_{whole}(k) &= \frac{E * m}{k} [t_{iter}(k) + \Delta] \\ &= \frac{E * m}{k} \left[\sum_{j=1}^{2N} \frac{\frac{k}{k_{base}} FLOPs_j(base)}{FLOPS(\frac{k}{k_{base}} FLOPs_j(base))} + \Delta \right] \end{aligned} \quad (3)$$

where Δ denotes the time expenditure between two iterations, in single GPU mode, it is negligible with pre-cached input samples, however, additional time for parameter synchronization among cluster in distributed mode must be taken into account. Noting that there are two parts in $t_{whole}(k)$. The

first part is pure GPU computation time, with the increase of minibatch size k , denominator FLOPS(*) also increases until saturation, thus, the computation time gets shorter and levels off finally. Value of minibatch size k at threshold point hinges on specific deep neural network and hardware resource. The second part makes sense chiefly in distributed mode, especially for those networks with massive model parameters, reducing times for parameter synchronization by increasing minibatch size k is the most straightforward and effective method.

GPU memory usage model. We take two independent streams, $\text{stream}_{\text{swap-out}}$ and $\text{stream}_{\text{swap-in}}$, to execute the offload and prefetch operations in background. In our design, an active area with size of peak memory usage is assigned in GPU memory to serve cyclically for per layer's computation. In the layer-wise propagation, peak memory usage can be achieved within finite times even only once, thus, the active area is underutilized in most cases. This fact provides $\text{stream}_{\text{swap-in}}$ with opportunity to swap in subsequent data of other layers as soon as possible, which is helpful to ease GPU computation stall problem. With allocate operation consuming GPU memory and release operation freeing memory concurrently, the real-time memory usage must not be greater than the given GPU memory budget M_{budget} . Then, at any time, the finished subsequences $\text{Seq}_{\text{allocate}}$ and $\text{Seq}_{\text{release}}$ satisfy constraint (4), in which data size of each operation $M_{\text{allocate/release}}(k)$ is obtained with static information 2).

$$\sum_{p=0}^{\text{Seq}_{\text{allocate}}} \frac{k}{k_{\text{base}}} M_{\text{allocate}[p]}(k_{\text{base}}) - \sum_{q=0}^{\text{Seq}_{\text{release}}} \frac{k}{k_{\text{base}}} M_{\text{release}[q]}(k_{\text{base}}) \leq M_{\text{budget}} \quad (4)$$

PCIe communication model. PCIe gen3 provides a maximum data transfer bandwidth of 16 GB/s between host and GPU device. By averaging a group of actual transfer rates that derived from static information 2) and dynamic information 2), a more down-to-earth PCIe bandwidth $\text{Bandwidth}_{\text{avail}}$ in current hardware environment is obtained. The runtime of prefetch/offload operation under minibatch size k is roughly formulated as (5) where $T_{\text{pre/off}}(*)$ represents the data size to be transferred. If a feature map is pinned in GPU memory, $t_{\text{pre/off}} = 0$.

$$t_{\text{pre/off}} = \frac{\frac{k}{k_{\text{base}}} T_{\text{pre/off}}(k_{\text{base}})}{\text{Bandwidth}_{\text{avail}}} \quad (5)$$

D. Constraint condition

The above three submodels are interrelated and mutually restricted, impacting the performance of swap out/in strategy together. To avoid stalling GPU computation, we define a time sequence t_{ready} with length of $2N$ in the range of a single iteration, in which each element identifies a timestamp after when layer j can start computing at any moment, as mentioned in section A. Algorithm 1 demonstrates the key steps of determining t_{ready} .

Additional remarks for Algorithm 1. Because the time cost for memory allocation operation is trivial, we primarily consider the prefetch operation here. On condition of trainability, the involved data of first layer can be prefetched directly at the beginning of iteration. If GPU memory is sufficient, prefetch operations of subsequent layers will run continuously without interruption, $t_{\text{ready}}[j]$ equals to the accumulation of communication time. Once no memory is available, prefetch operation will be suspended until enough memory space is released. According to the interrupted timestamp, we can estimate layer j_{current} that $\text{stream}_{\text{compute}}$ is executing now, then, by traversing $\text{Seq}_{\text{release}}$ from j_{current} to $j_{\text{satisfied}}$ until the memory to be released in this range satisfies the requirement of the suspended layer, $j_{\text{satisfied}}$ is picked out. So, the corresponding $t_{\text{ready}}[j]$ equals to the sum of computation time from layer 1 to $j_{\text{satisfied}}$ and the prefetch communication time. Notice that the real-time memory usage $M_{\text{used_present}}$ always increases in Algorithm 1, actually, with the proceeding of computation, memory release operations conducted by $\text{stream}_{\text{swap-out}}$ also decrease $M_{\text{used_present}}$, we just don't depict it here explicitly.

Algorithm 1: Determining the time sequence t_{ready}

Input: subsequences $\text{Seq}_{\text{allocate}}$ and $\text{Seq}_{\text{release}}$, GPU memory budget M_{budget}

Output: time sequence t_{ready}

1. $t_{\text{ready}}[j] = 0$, j is the layer index;
 2. $M_{\text{used_present}} = 0$;
 3. for ($j = 1$; $j \leq 2N$; $j++$) {
 4. if ($j == 1$) {
 5. $t_{\text{ready}}[j] = \sum_j t_{\text{pre}}$ sum of t_{pre} for layer j 's prefetch operations;
 6. } else if ($M_{\text{used_present}} + \text{size of Seq}_{\text{allocate}}^j \leq M_{\text{budget}}$) {
 7. $t_{\text{ready}}[j] = t_{\text{ready}}[j-1] + \sum_j t_{\text{pre}}$;
 8. } else {
 9. Estimate current layer j_{current} be executing by comparing $t_{\text{ready}}[j-1]$ and $\sum_1^{\text{current}} t_{\text{compute}}$;
 10. Traverse $\text{Seq}_{\text{release}}$ from layer j_{current} to $j_{\text{satisfied}}$ until find $j_{\text{satisfied}}$ that satisfies sum of the released memory from layer j_{current} to $j_{\text{satisfied}}$ larger than size of $\text{Seq}_{\text{allocate}}^j$;
 11. $t_{\text{ready}}[j] = \sum_1^{\text{satisfied}} t_{\text{compute}} + \sum_j t_{\text{pre}}$;
 12. }
 13. $M_{\text{used_present}} += \text{size of Seq}_{\text{allocate}}^j$;
 14. }
 15. return t_{ready} ;
-

With t_{ready} , the constraint of without stalling GPU computation is expressed as

$$t_{\text{ready}}[j] \leq \sum_{r=1}^{j-1} t_r \quad (6)$$

The ready timestamp of layer j must be ahead of the accomplishment of layer $(j-1)$'s computation.

V. HYPERPARAMETER TUNER

This section elaborates on solving the efficiency-optimal minibatch size and the matched learning rate in the new runtime environment with host memory and PCIe communication involved after applying the dynamic swapping strategy. By integrating the training time and constraints of swapping strategy as a general optimization problem, the efficiency-optimal minibatch size and its corresponding swapping strategy are jointly solved. Besides, to guarantee the final accuracy, we figure out an adaptive rule of the learning rate to accommodate the efficiency-optimal minibatch size.

Algorithm 2: Determining the efficiency-optimal minibatch size

Input: GMAP, GPU memory budget M_{budget}

Output: Efficiency-optimal minibatch size k^* and its corresponding swapping strategy

1. Traverse GMAP to get the Seq_{peak} when achieve peak memory usage;
 2. $k_{max} = k_{base} * [M_{budget} - M_{others} - M_{para}(\text{in } Seq_{peak}) - M_{ws}(\text{in } Seq_{peak})] / \sum [M_{FM}(k_{base})(\text{in } Seq_{peak})]$;
 3. $k = k_{max}$;
 4. Calculate t_{ready} under k with constraint (4);
 5. while (constraint (6) is not satisfied under k for all layer 1 to $2N$) {
 6. $k -= 1$;
 7. Calculate t_{ready} under k with $M_{budget} = M_{peak}(k)$ in constraint (4);
 8. Identify layer index set Ω that don't satisfy (6);
 9. Try to pin data involved in Ω in GPU memory area ($M_{budget} - M_{peak}(k)$);
 10. }
 11. $k^* = k$;
 12. return the efficiency-optimal minibatch size k^* ;
-

A. Efficiency-optimal minibatch size

In the light of performance model and the constraint, the whole training process with dynamic swap out/in strategy can be formulated as an optimization problem (7) with respect to minibatch size

$$\begin{aligned} \min \quad & (3) \\ \text{s.t.} \quad & (4) \ \& \ (6) \end{aligned} \quad (7)$$

Equation (7) can be categorized as an integer linear programming problem, we build a linear search algorithm to determine the efficiency-optimal minibatch size. See Algorithm 2. The search process starts with the maximum minibatch size and drops to the efficiency-optimal point where the two constraints are both satisfied. At the same time, which feature map to be swapped or pinned under the efficiency-optimal minibatch size is also identified.

B. Adaptive learning rate

After determining the efficiency-optimal minibatch size from a systematic perspective, the adaptive learning rate should also be taken into account as algorithmic improvement. Only by the co-design of system and algorithm can we achieve both satisfying training time and accuracy. From practical experience, with a constant number of epochs, larger minibatch size usually leads to final accuracy loss if maintaining the same learning rate with those small minibatch sizes. Hence, we try to overcome this barrier by drawing on the idea in [17] to adjust learning rate adaptively. Theorem 4.6 in [17] demonstrates the convergence property with fixed learning rate. To retain the same convergence rate under different minibatch size, the contraction item should meet

$$(1 - \alpha_{base} c \mu)^{iters_{base} - 1} = (1 - \alpha^* c \mu)^{\frac{iters_{base} - 1}{q} - 1} \quad (8)$$

In (8), α_{base} and $iters_{base}$ denote learning rate and iterations under minibatch size k_{base} while α^* is the learning rate to be solved that matches with k^* . q is multiple between k^* and k_{base} . The gradient of large minibatch size is relatively close to unbiased estimate, then $\mu \approx 1$. c represents convexity of objective function. By solving (8),

$$\alpha^* \approx \frac{1 - (1 - \alpha_{base} c)^q}{c} \quad (9)$$

With larger minibatch size, the reduction of variance of stochastic gradient direction allows a bigger step in each iteration, compensating for deficiency of fewer parameter update times in an alternative way.

VI. RUNTIME MEMORY MANAGER

The runtime memory manager provides four basic memory access operations: allocate, release, offload, prefetch. Allocate/release operation includes memory allocation and release in both GPU and host side. In GPU side, we first allocate all the given memory budget as an integrated buffer pool with one `cudaMalloc()` call when a new iteration starts, the memory manager then allocates GPU buffer from the buffer pool directly for each allocate operation at runtime, accordingly, the release operation returns the buffer space back to the buffer pool. The memory space in host side is allocated in pinned way, removing the time cost of the copy step between pageable and pinned memory area. Offload/prefetch operation copies the required data between GPU and host memory by PCIe communication.

To fully overlap the GPU computation and communication of swapping operation, the runtime memory manager deploys three separate CUDA streams for computation, swap-out and swap-in. Such a design is well-matched to most of GPU architectures with a kernel engine and two copy engines (Device-to-Host copy engine and Host-to-Device copy engine). `streamswap-in` manages the allocate and prefetch operations while `streamswap-out` manages the release and offload operations. For swap-out operation, `streamswap-out` waits for

notification from `streamcompute`, once a feature map is generated, `streamswap-out` starts offload operation immediately. For swap-in operation, `streamswap-in` prefetches successively until no GPU memory is available and the interrupt will be recovered after `streamswap-out` releases some preceding memory occupancies. The offload operation mostly takes place in the forward propagation phase and the prefetch operation corresponding to the backward propagation phase. Offload as soon as possible is beneficial to spare more memory space for posterior executions, likewise, prefetch early can avert stalling GPU computation as far as possible.

VII. EXPERIMENT

A. Experimental setup

AccUDNN is evaluated on those state-of-art models in ILSVRC classification task, particularly the UDNN ResNet and its variants. Each server has one NVIDIA M40 GPU with 24GB GPU device memory and 128GB RAM. Eight servers interconnected with 10Gbps Ethernet constitute the distributed cluster. We implement AccUDNN on the open-source framework Caffe with slight modification. The overall training procedure is split into three stages. We begin with the profiling stage in the first five epochs with user-specified learning rate, then, a decision-making process is conducted with the established performance model to single out the efficiency-optimal minibatch size, corresponding swapping strategy, and the matched learning rate, finally, run the remaining epochs with the optimal configuration till the end.

B. Evaluations of AccUDNN

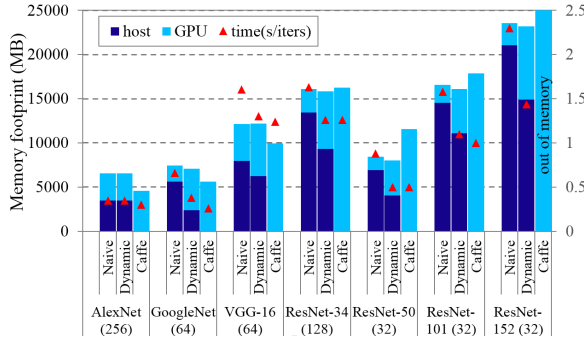


Fig. 4: Evaluation of dynamic swap out/in strategy

Dynamic swap out/in strategy. Figure 4 presents the memory footprint for seven well-known UDNNs. Three comparative cases are studied here. The “naive” refers to the naive swap out/in strategy crudely offload all the feature maps to host memory. The “dynamic” refers to our dynamic swapping strategy proposed in the memory optimizer. The “Caffe” refers to the original Caffe that keeps all the data in GPU.

It shows that compared with the original Caffe, naive strategy greatly reduces the trainable GPU memory requirement. Typically, for the ResNet-152 with minibatch size 32, from more than 24GB to 2.5GB, the swap out/in strategy shrinks the

GPU memory requirement from network-wide level to layer-wise level successfully. However, the minimum trainable GPU memory capacity stalls the computation which leads to an undesirable increase of training time (the red triangle). Our dynamic strategy achieves a moderate tradeoff by increasing a portion of GPU memory and maintains a nearly identical time cost with original Caffe. Particularly, among those seven neural networks with incremental depth, the swapping strategy is more beneficial to UDNN, as ResNet-152, almost 64% involved feature maps are offloaded to host memory and only 8.3GB GPU memory is required, making training UDNNs on most of the mid to low-end GPUs possible.

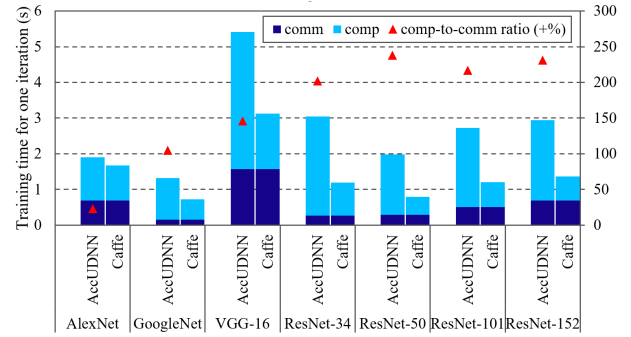


Fig. 5: Performance in distributed cluster

Efficiency-optimal hyperparameter. Given 12GB GPU memory budget, the efficiency-optimal minibatch size after applying the dynamic swapping strategy increases by 22-315% and keeps almost the same throughput with original Caffe, as shown in Table I. Likewise, the improvement for UDNN is more notable than other mid-depth networks on account of dramatical memory saving from network-wide to layer-wise. Distinctively, the training throughput grows 17% and 26% for ResNet-101 and ResNet-152 respectively because the Caffe’s memory-restricted minibatch size is not enough to invoke all the GPU computing resources. After getting the optimal minibatch size, we further validate the effectiveness of the adaptive learning rate and obtain the results in Table II.

Scaling to distributed cluster. We adopt the parameter server architecture [18] to run the distributed training task. By deploying AccUDNN accelerator on every individual GPU and setting minibatch size as the efficiency-optimal value in Table I, we get results in Figure 5. The computation-to-communication ratio (the red triangle) of AccUDNN stands in stark contrast to original Caffe-PS. For ResNet-152, the training time is saved about 48%.

For ResNet-101, AccUDNN achieves 7.2x speed-up with 8 GPUs, surpassing Caffe-PS apparently. Due to underutilization of GPU computing resource in Caffe default mode, AccUDNN gets the benefit from both the efficiency improvement on single GPU and reduction of communication overhead among the cluster. Without any communication optimization, we have achieved a satisfying scaling efficiency. Details are listed in Table III.

TABLE I: Effectiveness of minibatch size selection

	AccUDNN		Caffe		Minibatch size (+%)
	Efficiency-optimal minibatch	Throughput (images/s)	Max trainable minibatch	Throughput (images/s)	
AlexNet	1032	853	848	865	22
GoogleNet	286	245	150	263	91
VGG-16	192	50	80	51	140
ResNet-34	276	99	91	99	203
ResNet-50	112	66	33	66	239
ResNet-101	74	34	20	29	270
ResNet-152	54	24	13	19	315

TABLE II: Model accuracy on ImageNet classification task

	Efficiency-optimal minibatch	Initial learning rate	Top-1 accuracy
AlexNet	1032*8	0.318	57.9
GoogleNet	286*8	0.69	67.8
ResNet-50	112*8	0.345	75
ResNet-101	74*8	0.233	76.3

Comparison with related work. Recent work SuperNeurons [8] couples the swapping technique with recomputation mechanism to tackle the trainability issues of UDNN. They finally achieve a GPU memory-optimal state and enable much larger minibatch size to be processed, but the training efficiency is affected negatively when realizing memory-optimal. The improvement of AccUDNN lies in the shift from memory-optimal to efficiency-optimal, we are not limited to memory itself. The comparisons are shown in Table IV. The efficiency-optimal minibatch size achieved by AccUDNN is 54-300% larger than SuperNeurons's and the corresponding throughput also increase 10-106%.

VIII. CONCLUSION

In this paper, we propose AccUDNN to accelerate the training process of UDNN from the perspective of GPU memory optimization. By applying the performance-model guided dynamic swap out/in strategy, memory optimizer offloads those suitable feature maps to host memory, thereby breaking the trainability restriction in a way without incurring performance degradation. Then, hyperparameter tuner is adopted to explore the efficiency-optimal hyperparameter setting after applying the dynamic swapping strategy so as to further accelerate the training process. Finally, evaluations against the state-of-the-art DL framework Caffe have demonstrated the effectiveness of AccUDNN.

ACKNOWLEDGEMENT

This research is supported by the National Key Research and Development Program of China (No. 2017YFB1010000).

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, pp. 1097–1105, December 2012.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, June 2016.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

TABLE III: Acceleration for UDNN ResNet-101/152

		GPUs	Minibatch size	Throughput (images/s)	Scaling efficiency
Resnet-101	Caffe	1	20	30	1
	Caffe_PS	8	20 * 8 = 160	132	4.4
	AccUDNN_PS	8	74 * 8 = 592	217	7.2
Resnet-152	Caffe	1	13	19	1
	Caffe_PS	8	13 * 8 = 104	76	4
	AccUDNN_PS	8	54 * 8 = 432	147	7.7

TABLE IV: Comparison with SuperNeurons

	Efficiency-optimal minibatch size			Corresponding throughput		
	SuperNeurons	AccUDNN	(+%)	SuperNeurons	AccUDNN	(+%)
AlexNet	640	1032	61	414	853	106
VGG-16	48	192	300	30	50	67
ResNet-50	48	112	133	60	66	10
ResNet-101	48	74	54	30	34	13
ResNet-152	32	54	68	17	24	42

- [4] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, "Xlnet: Generalized autoregressive pretraining for language understanding," *arXiv preprint arXiv:1906.08237*, 2019.
- [5] Y. You, Z. Zhang, C.-J. Hsieh, J. Demmel, and K. Keutzer, "100-epoch imagenet training with alexnet in 24 minutes," *ArXiv e-prints*, 2017.
- [6] A. A. Awan, K. Hamidouche, J. M. Hashmi, and D. K. Panda, "S-caffe: co-designing mpi runtimes and caffe for scalable deep learning on modern gpu clusters," *Acm Sigplan Notices*, vol. 52, no. 8, pp. 193–205, 2017.
- [7] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfikar, and S. W. Keckler, "vdsn: Virtualized deep neural networks for scalable, memory-efficient neural network design," in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, p. 18, IEEE Press, October 2016.
- [8] L. Wang, J. Ye, Y. Zhao, W. Wu, A. Li, S. L. Song, Z. Xu, and T. Kraska, "Superneurons: dynamic gpu memory management for training deep neural networks," in *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 41–53, ACM, February 2018.
- [9] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.
- [10] A. Jain, A. Phanishayee, J. Mars, L. Tang, and G. Pekhimenko, "Gist: Efficient data encoding for deep neural network training," in *ACM/IEEE 45th Annual International Symposium on Computer Architecture*, June 2018.
- [11] C. Meng, M. Sun, J. Yang, M. Qiu, and Y. Gu, "Training deeper models by gpu memory optimization on tensorflow," in *Proc. of ML Systems Workshop in NIPS*, December 2017.
- [12] J. Guo, W. Liu, W. Wang, Q. Lu, S. Hu, J. Han, and R. Li, "A gpu memory efficient speed-up scheme for training ultra-deep neural networks: poster," in *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, February 2019.
- [13] T. Akiba, S. Suzuki, and K. Fukuda, "Extremely large minibatch sgd: Training resnet-50 on imagenet in 15 minutes," *arXiv preprint arXiv:1711.04325*, 2017.
- [14] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "On parallelizability of stochastic gradient descent for speech dnns," in *Acoustics, Speech and Signal Processing (ICASSP), 2014 IEEE International Conference on*, pp. 235–239, IEEE, May 2014.
- [15] S. Chetlur, C. Woolley, P. Vandermers, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014.
- [16] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: training imagenet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.
- [17] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018.
- [18] H. Cui, H. Zhang, G. R. Ganger, P. B. Gibbons, and E. P. Xing, "Geeps: Scalable deep learning on distributed gpus with a gpu-specialized parameter server," in *Proceedings of the Eleventh European Conference on Computer Systems*, p. 4, ACM, April 2016.